

Documentação Técnica — Sistema de Tratativa de Reservas Não Confirmadas

1. Objetivo do Projeto

O sistema foi desenvolvido para auxiliar no tratamento e acompanhamento de reservas não confirmadas, permitindo a consulta de informações da reserva e o registro da tratativa realizada pela equipe operacional.

O objetivo principal é centralizar o histórico das ações tomadas sobre cada reserva e facilitar o acompanhamento dos casos pendentes.

2. Arquitetura do Projeto

O projeto segue uma arquitetura separada entre frontend e backend:

```
projeto/  
├── frontend/      → Aplicação Angular  
└── backend/       → API Python com FastAPI
```

Frontend

Responsável pela interface do usuário e interação com a API.

Backend

Responsável pelas regras de negócio, comunicação com banco de dados e disponibilização dos endpoints.

Banco de Dados

Responsável pelo armazenamento das reservas e das tratativas realizadas.

3. Tecnologias Utilizadas

Frontend

- Angular 18
- TypeScript
- HTML5
- SCSS

- Angular Forms (FormsModule)
- Angular Material
 - MatSelect
 - MatFormField
 - MatInput
- SweetAlert2
- Bootstrap Icons

Backend

- Python 3
- FastAPI
- Uvicorn
- Pydantic
- Psycpg2

Banco de Dados

- PostgreSQL
 - pgAdmin 4
-

4. Estrutura do Frontend

HomeComponent

Responsável pelo gerenciamento do estado principal da aplicação.

Funções:

- Receber a reserva pesquisada.
 - Receber a tratativa associada.
 - Limpar os dados exibidos quando solicitado pelo usuário.
-

HeaderComponent

Componente responsável pelo cabeçalho da aplicação.

InputComponent

Responsável pela pesquisa de reservas.

Funcionalidades:

- Busca da reserva pelo número informado.
- Consulta da tratativa existente.
- Exibição de mensagens de erro utilizando SweetAlert2.
- Botão para limpar os dados da tela.

Eventos emitidos:

- reservaEncontrada
 - limparDados
-

ReservationDetailsCardComponent

Responsável pela exibição das informações da reserva.

Dados exibidos:

- Número da reserva
 - Nome da empresa
 - Nome do requisitante
 - Agência
 - Nome do condutor
 - Grupo
 - E-mail
 - Telefone
 - Data de retirada
 - Data de devolução
-

ReservationTaskCardComponent

Responsável pelo cadastro e atualização das tratativas.

Campos disponíveis:

- Status
- Responsável
- Confirmada por
- Observações
- Novo localizador (quando aplicável)

Status disponíveis:

- Aberta

- Não Confirmada
- Duplicada
- Nova Reserva Criada

Caso o status seja “Nova Reserva Criada”, o campo “Novo Localizador” é exibido automaticamente.

5. Estrutura do Backend

A API foi desenvolvida utilizando FastAPI e possui comunicação direta com o PostgreSQL utilizando Psycopg2.

Responsabilidades

- Buscar reservas.
 - Buscar tratativas.
 - Salvar novas tratativas.
 - Atualizar tratativas existentes.
 - Validar dados recebidos.
-

6. Principais Endpoints

Buscar reserva

GET /reservas/{numero_reserva}

Retorna os dados da reserva pesquisada.

Buscar tratativa

GET /reservas_tratadas/{numero_reserva}

Retorna a tratativa associada à reserva.

Salvar ou atualizar tratativa

POST /reservas_tratadas

Realiza inserção ou atualização da tratativa da reserva.

7. Modelo de Dados

Tabela: reservas

Campos principais:

- id
 - numero_reserva
 - nome_empresa
 - nome_requisitante
 - agencia
 - nome_condutor
 - grupo
 - email
 - telefone
 - data_retirada
 - data_devolucao
 - status
-

Tabela: reservas_tratadas

Campos principais:

- id
 - numero_reserva
 - status_tratado
 - responsavel
 - confirmada_por
 - observacoes
 - numero_nova_reserva
-

8. Fluxo da Aplicação

Consulta

1. O usuário informa o número da reserva.
 2. O sistema consulta a API.
 3. Os dados da reserva são exibidos.
 4. O sistema consulta automaticamente a existência de uma tratativa.
-

Tratativa

1. O operador seleciona o status.
 2. Preenche os dados necessários.
 3. Salva a tratativa.
 4. O sistema cria ou atualiza o registro.
-

Nova Pesquisa

1. O usuário utiliza o botão de limpar.
 2. O campo de pesquisa é limpo.
 3. Os cards retornam ao estado inicial.
 4. Uma nova reserva pode ser pesquisada.
-

9. Tratamento de Erros

O sistema utiliza SweetAlert2 para exibição das mensagens.

Principais cenários:

Reserva não encontrada

Exibição de alerta informando que a reserva não foi localizada.

Erro interno

Exibição de mensagem genérica para falhas do servidor ou banco de dados.

Sucesso

Confirmação visual para inserção ou atualização da tratativa.

10. Configuração do Ambiente

Backend

Instalação das dependências:

```
pip install fastapi
pip install uvicorn
pip install psycpg2
pip install pydantic
```

Inicialização da API:

```
uvicorn main:app --reload
```

Frontend

Instalação das dependências:

```
npm install
```

Inicialização do projeto:

```
ng serve
```

11. Comunicação entre Serviços

O frontend e o backend se comunicam via requisições HTTP utilizando JSON.

Durante o desenvolvimento, ambos os serviços são executados localmente e a comunicação é autorizada através da configuração de CORS no backend.

12. Bibliotecas Externas Utilizadas

Frontend

- @angular/material
- sweetalert2
- bootstrap-icons

Backend

- fastapi
 - uvicorn
 - psycopg2
 - pydantic
-

13. Possíveis Melhorias Futuras

- Sistema de autenticação e autorização.
- Histórico completo de alterações das tratativas.
- Filtros por status.
- Dashboard com métricas operacionais.
- Paginação das consultas.
- Pesquisa avançada por período.
- Controle de usuários responsáveis pelas alterações.
- Deploy em ambiente de produção.